MODUL 4

Clustering I (KMeans)

A. Tujuan Praktikum

- Memahami konsep dasar algoritma K-Means Clustering.
- Mampu mengimplementasikan K-Means menggunakan Python.
- Mampu melakukan visualisasi hasil clustering.
- Menginterpretasikan hasil clustering untuk pengambilan keputusan.
- Mengevaluasi kualitas cluster menggunakan Silhouette Score.

B. Alokasi Waktu

1 x pertemuan = 120 menit

C. Dasar Teori

1. Definisi Clustering

Clustering adalah metode *unsupervised learning* yang bertujuan mengelompokkan data ke dalam beberapa kelompok (cluster) berdasarkan kemiripan karakteristiknya, tanpa menggunakan label target. Ini berbeda dengan *supervised learning*, di mana algoritma belajar dari data yang sudah diberi label jawaban.

Sederhananya, data yang mirip satu sama lain dikelompokkan ke *cluster* yang sama, sedangkan data yang berbeda karakteristiknya dipisahkan ke *cluster* lain. Misalnya, jika kita punya data pelanggan tanpa keterangan jenis pelanggan apa pun, *clustering* dapat menemukan sendiri bahwa ternyata ada kelompok pelanggan dengan pendapatan tinggi dan belanja tinggi, serta kelompok lain dengan pendapatan rendah dan belanja rendah, tanpa kita perlu menentukan kelompok tersebut di awal.

Pengukuran kemiripan ini umumnya didasarkan pada perhitungan jarak, seperti jarak Euclidean, Manhattan, atau Cosine similarity, tergantung pada karakteristik data yang digunakan. *Clustering* banyak digunakan pada berbagai bidang, antara lain segmentasi pelanggan, pengelompokan dokumen, analisis citra, hingga deteksi anomali.

2. Konsep K-Means

K-Means adalah salah satu algoritma *clustering* yang paling populer, termasuk dalam kategori *partitioning clustering*, yaitu metode yang membagi data ke dalam sejumlah *cluster* (sebanyak K, ditentukan di awal) tanpa tumpang tindih antar *cluster*.

Sederhananya, bayangkan titik-titik data tersebar pada sebuah bidang. *K-Means* mencoba menemukan K titik pusat (centroid) yang paling representatif untuk mewakili kelompok-kelompok data di sekitarnya. Setiap titik data kemudian bergabung ke pusat terdekatnya. Proses ini dilakukan berulang kali, karena pada awalnya posisi pusat tersebut hanya tebakan awal, algoritma perlu menggeser posisi pusat sedikit demi sedikit sampai posisinya benar-benar mewakili kelompok data di sekitarnya.

2. 1 Centroid

Centroid adalah titik pusat *cluster*, dihitung sebagai rata-rata seluruh titik data yang tergabung dalam *cluster* tersebut. *Centroid* inilah acuan utama proses pengelompokan, setiap titik data akan dimasukkan ke *cluster* dengan *centroid* terdekat.

2. 2 Perhitungan Jarak

Untuk menentukan jarak terdekat, *K-Means* secara *default* menggunakan jarak *Euclidean* (jarak garis lurus antar dua titik). Karena jarak ini mengukur kedekatan secara menjari ke segala arah dari centroid, *K-Means* secara alami cenderung membentuk *cluster* berbentuk bulat dengan ukuran relatif seimbang. Akibatnya, jika data asli memiliki bentuk *cluster* yang memanjang, tidak beraturan, atau ukurannya jauh berbeda, *K-Means* bisa salah mengelompokkan data, inilah salah satu keterbatasan utama *K-Means*.

2. 3 Inisialisasi Centroid

Karena posisi *centroid* awal mengacak, cara menebak posisi awal ini berpengaruh terhadap hasil akhir, berikut beberapa cara inisialisasi *centroid*:

a. Acak: memilih K titik data secara acak sebagai *centroid* awal. Cara ini sederhana, tetapi jika posisi awal kebetulan berdekatan atau kurang mewakili, algoritma berisiko berhenti pada hasil pengelompokan yang kurang optimal.
b. *k-means*++: cara pemilihan *centroid* dengan cara ini ialah dengan memastikan titik-titik awal tersebar sejauh mungkin satu sama lain. Cara ini terbukti mempercepat proses penggabungan dan menghasilkan *cluster* yang lebih stabil, sehingga dijadikan metode *default* pada parameter *init* di *sklearn.cluster.KMeans*.

2. 4 Fungsi Objektif dan Langkah Praktikum

Supaya proses menggeser *centroid* punya arah yang jelas, *K-Means* membutuhkan sebuah ukuran yang menyatakan seberapa baik suatu pengelompokan. Ukuran ini disebut *Within-Cluster Sum of Squares* (WCSS), yaitu total jarak kuadrat antara setiap titik data dengan *centroid cluster*-nya masing-masing.

Tujuan dari penggunaan WCSS ini ialah semakin kecil nilai WCSS, berarti titik-titik data dalam tiap *cluster* semakin rapat/dekat dengan *centroid*-nya, yang menandakan *cluster* tersebut semakin kompak dan baik. Karena itu, tujuan *K-Means* adalah mencari posisi *centroid* yang meminimalkan nilai WCSS. Nilai WCSS ini pula yang nantinya dipakai kembali pada bagian Evaluasi *Cluster* untuk membantu menentukan jumlah *cluster* yang paling tepat.

Secara Matematis, WCSS dirumuskan sebagai:

$$\underset{C}{\operatorname{argmin}} \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

dengan *Ck* adalah himpunan data pada *cluster* ke-*k*, dan μ*k* adalah *centroid* dari *cluster* tersebut.

Berdasarkan tujuan meminimalkan WCSS tersebut, proses *K-Means* dijalankan secara berulang melalui langkah-langkah berikut, sampai posisi *centroid* stabil (tidak berubah signifikan) atau mencapai batas iterasi maksimum. Berikut langkah algoritmanya:

1. Tentukan K (jumlah *cluster*).
2. Inisialisasi *centroid* awal (pilih K titik secara acak atau pakai *k-means++*).
3. *Assignment step*: untuk setiap titik x, beri label *cluster = argmin_k distance*(*x, centroid_k*).
4. *Update step*: hitung ulang *centroid* μ*k* sebagai rata-rata semua titik yang ditugaskan ke *cluster* k.
5. Ulangi langkah 3–4 sampai *centroid* tidak berubah signifikan (konvergen) atau mencapai iterasi maksimum.

3. Kelebihan dan Kekurangan

Kelebihan:

a. Sederhana, mudah dipahami, dan cepat, kompleksitas komputasinya linear terhadap jumlah data (waktu proses naik sebanding jumlah data, bukan berlipat ganda), sehingga cocok untuk dataset besar.

b. Tersedia luas di berbagai *library* (scikit-learn, Spark MLlib) dan dapat diterapkan di banyak bidang.

c. Hasil representatif jika distribusi data relatif bulat/terpisah jelas.

Kekurangan:

a. Jumlah *cluster* (K) harus ditentukan di awal, padahal sering sulit diperkirakan tanpa eksplorasi data lebih dulu.
b. Sensitif terhadap *outlier*, karena *outlier* ikut memengaruhi posisi *centroid* saat dihitung sebagai rata-rata, satu titik yang sangat jauh dapat menarik *centroid* menjauh dari sebagian besar data.
c. Mengasumsikan *cluster* berbentuk bulat, kurang cocok untuk pola *cluster* yang memanjang atau tidak beraturan.
d. Rentan terjebak solusi lokal, karena hasil akhir bergantung pada posisi *centroid* awal.
e. Hanya dapat mengolah data numerik, karena *centroid* dihitung dari rata-rata nilai, data kategorikal perlu diubah terlebih dahulu.

4. Evaluasi Cluster

Setelah proses *clustering* selesai, kita perlu cara untuk menilai apakah hasil pengelompokan tersebut sudah baik dan jumlah *cluster* yang dipilih sudah tepat. Dua metode yang umum digunakan adalah *Elbow Method* dan *Silhouette Score*.

4. 1 Elbow Method

Digunakan untuk membantu menentukan K optimal, dengan memanfaatkan nilai WCSS yang sudah dijelaskan pada bagian 2.4. Cara kerja dari *Elbow Method* dimulai dari menjalankan *K-Means* untuk beberapa nilai K berbeda, catat nilai *inertia* masing-masing, lalu plot hasilnya dalam grafik.

Nilai WCSS akan terus menurun seiring bertambahnya K, semakin banyak *cluster*, semakin dekat tiap titik ke *centroid*-nya. Namun, pada grafik tersebut penurunannya akan terlihat melandai tajam pada titik tertentu, membentuk pola seperti siku tangan (ini alasan mengapai dinamai sebagai *Elbow Method*). Nilai K pada titik itulah yang dipilih sebagai K optimal, karena penambahan *cluster* setelah titik ini tidak lagi memberikan perbaikan kualitas yang berarti.

4. 2 Silhouette Score

Jika *Elbow Method* membantu memilih kandidat nilai K, *Silhouette Score* membantu memvalidasi seberapa baik kualitas pemisahan antar *cluster* pada nilai K tersebut. Metode ini mengukur, untuk setiap titik data, seberapa mirip titik tersebut dengan *cluster*-nya sendiri dibandingkan dengan *cluster* lain yang terdekat.

Nilai *Silhouette Score* berkisar antara -1 hingga 1 dengan keterangan:

a. Mendekati 1: titik data sangat sesuai dengan *cluster*-nya (jauh lebih dekat ke *cluster* sendiri dibanding *cluster* lain).
b. Mendekati 0: titik berada di perbatasan antara dua *cluster*, sulit dikatakan *cluster* mana yang lebih cocok.
c. Negatif (< 0): titik kemungkinan salah tempat, karena justru lebih dekat ke *cluster* lain dibanding *cluster* tempatnya berada saat ini.

Semakin tinggi rata-rata *Silhouette Score* dari seluruh titik data, semakin baik kualitas pemisahan *cluster* yang dihasilkan.

D. Langkah Praktikum

1. Persiapan Environment

a. Buat *virtual environment Python* atau gunakan *Jupyter Notebook*.
b. Install *library* yang diperlukan:

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

2. Import Library

Tambahkan *library* yang akan digunakan:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris, make_moons
from sklearn.metrics import silhouette_score
```

3. Pengenalan Dataset

Pada praktikum ini, gunakan dataset *Iris* yang tersedia pada *scikit-learn*. Dataset ini berisi data panjang dan lebar sepal serta petal dari tiga jenis bunga Iris.

```
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
print(df.head())
```

Selain data fitur, dataset Iris juga menyediakan label spesies asli (setosa, versicolor, virginica). Label ini tidak akan digunakan saat proses *clustering* berlangsung (karena *K-Means* bersifat *unsupervised*), tetapi akan dipakai nanti pada tahap Interpretasi Hasil untuk membandingkan hasil *cluster* dengan kelompok aslinya.

```
species = data.target
species_names = data.target_names
print(species_names)
```

4. Standarisasi Data

Lakukan standardisasi agar semua fitur berada pada skala yang sama.

```
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

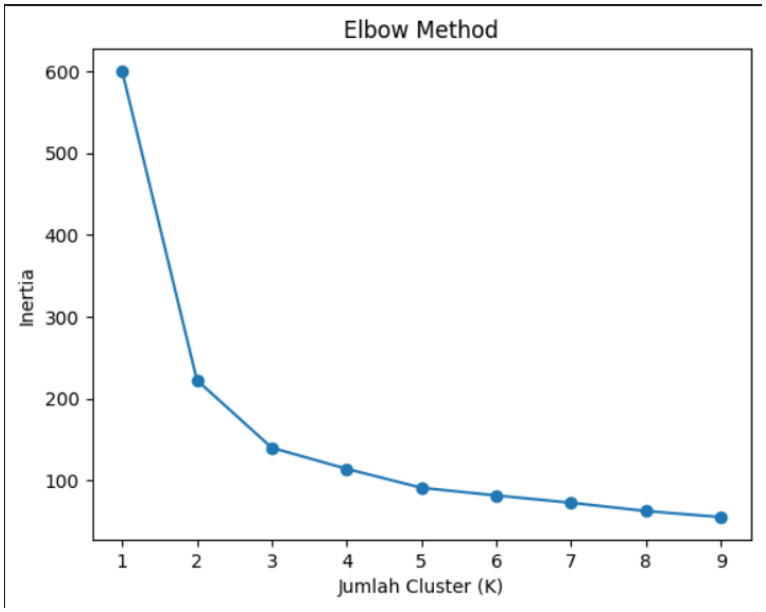
5. Menentukan Jumlah Cluster dengan Elbow Method

Hitung nilai *inertia* (WCSS) untuk berbagai nilai K.

```
inertia = []
K_range = range(1, 10)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.xlabel('Jumlah Cluster (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

K optimal belum diketahui di awal, sehingga beberapa kandidat nilai K dicoba sekaligus. Setiap K menghasilkan posisi *centroid* dan nilai WCSS yang berbeda, sehingga model perlu dilatih ulang untuk tiap K, hasil satu K tidak bisa dipakai untuk K lainnya. Nilai *inertia* tiap percobaan disimpan ke list terlebih dahulu karena pola siku baru terlihat jika seluruh nilai K dibandingkan bersama dalam satu grafik, bukan dilihat satu per satu.



Dari grafik *Elbow*, tentukan jumlah *cluster* optimal.

6. Implementasi K-Means

Setelah jumlah cluster dipilih, jalankan algoritma K-Means.

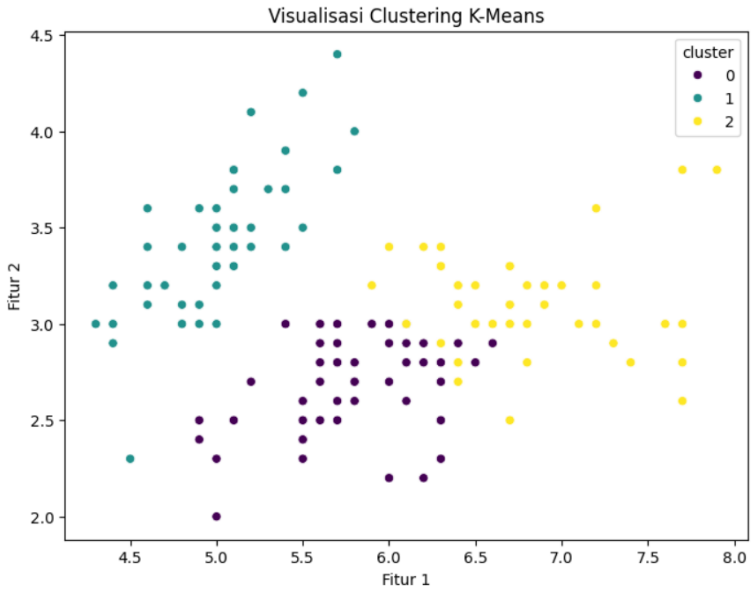
```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(df_scaled)
df['cluster'] = labels
print(df.head())
```

7. Visualisasi Hasil Clustering dengan Scatterplot

Visualisasikan data untuk melihat pembagian *cluster*.

Catatan: proses clustering dilakukan pada data yang sudah distandarisasi (df_scaled), namun visualisasi berikut menampilkan skala data asli (df) agar sumbu lebih mudah dibaca dalam satuan aslinya (cm).

```
plt.figure(figsize=(8,6))
sns.scatterplot(x=df.iloc[:,0],y=df.iloc[:,1], hue=df['cluster'],
palette='viridis')
plt.xlabel('Fitur 1')
plt.ylabel('Fitur 2')
plt.title('Visualisasi Clustering K-Means')
plt.show()
```



8. Evaluasi Cluster dengan Silhouette Score

Selain menggunakan *Elbow Method*, kualitas *cluster* juga dapat dievaluasi menggunakan *Silhouette Score*, yang mengukur seberapa mirip suatu titik dengan *cluster*-nya sendiri dibandingkan dengan *cluster* lain (lihat Dasar Teori bagian 4.2).

```
score = silhouette_score(df_scaled, labels)
print("Silhouette Score:", score)
```

9. Interpretasi Hasil

Sebelum menyimpulkan secara kualitatif, bandingkan dulu hasil *cluster* dengan label spesies asli yang sudah disiapkan pada Langkah 3:

```
comparison = pd.crosstab(df['cluster'], species,
rownames=['Cluster'], colnames=['Spesies Asli'])
comparison.columns = species_names
print(comparison)
```

Dengan mengacu pada tabel di atas dan nilai *Silhouette Score* dari Langkah 8, jawab pertanyaan berikut:

a. Amati pembagian cluster yang terbentuk.

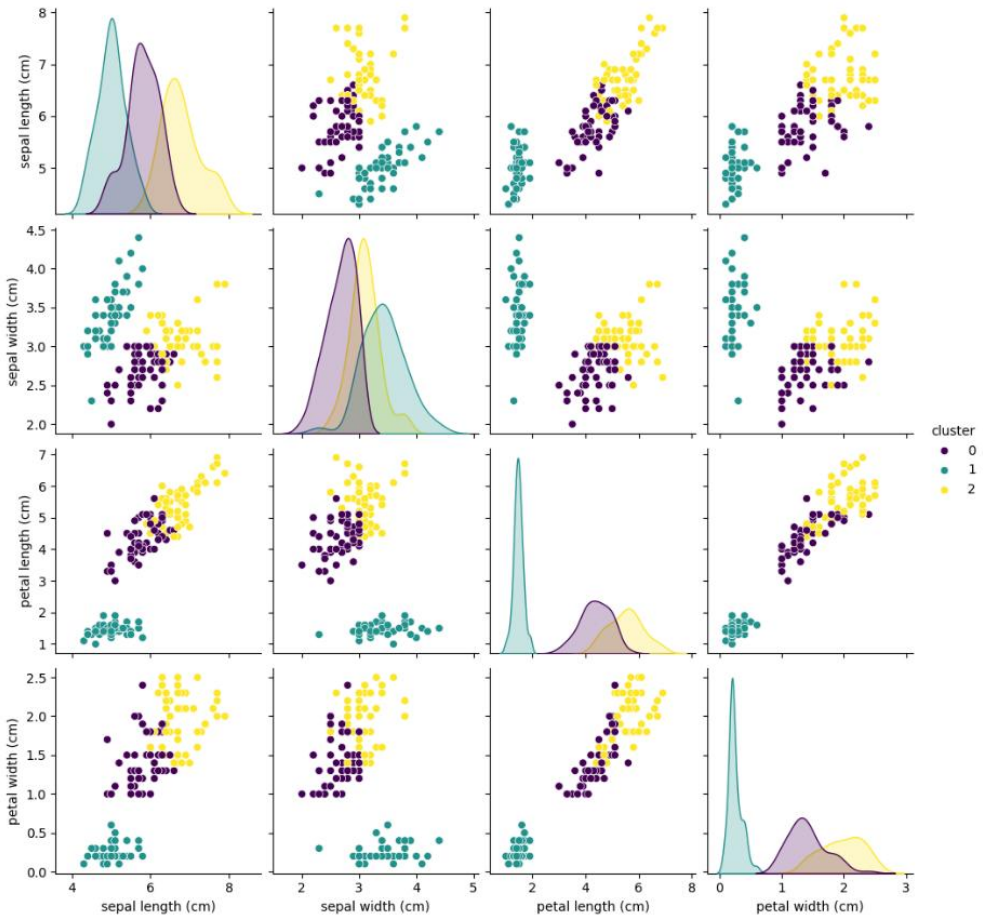
b. Apakah hasil cluster sesuai dengan label asli *Iris* (setosa, versicolor, virginica)? Spesies mana yang paling sering tertukar antar *cluster*, jika ada?

c. Bagaimana distribusi data di tiap *cluster*?

10. Visualisasi Cluster dengan Pairplot

Scatter plot hanya menampilkan 2 fitur sekaligus. Untuk dataset dengan lebih banyak fitur, kita bisa menggunakan *seaborn pairplot* agar semua fitur bisa divisualisasikan berpasangan.

```
sns.pairplot(df, hue='cluster', palette='viridis')
plt.show()
```



Interpretasi:

a. *Pairplot* menampilkan distribusi tiap fitur dalam bentuk histogram (di diagonal), dan hubungan antar fitur dalam *scatterplot*.

b. Warna berbeda menunjukkan cluster berbeda.

c. Dengan *pairplot*, kita bisa melihat fitur mana yang paling memisahkan *cluster*.

11. Bukti Keterbatasan K-Means pada Data Non-Spherical

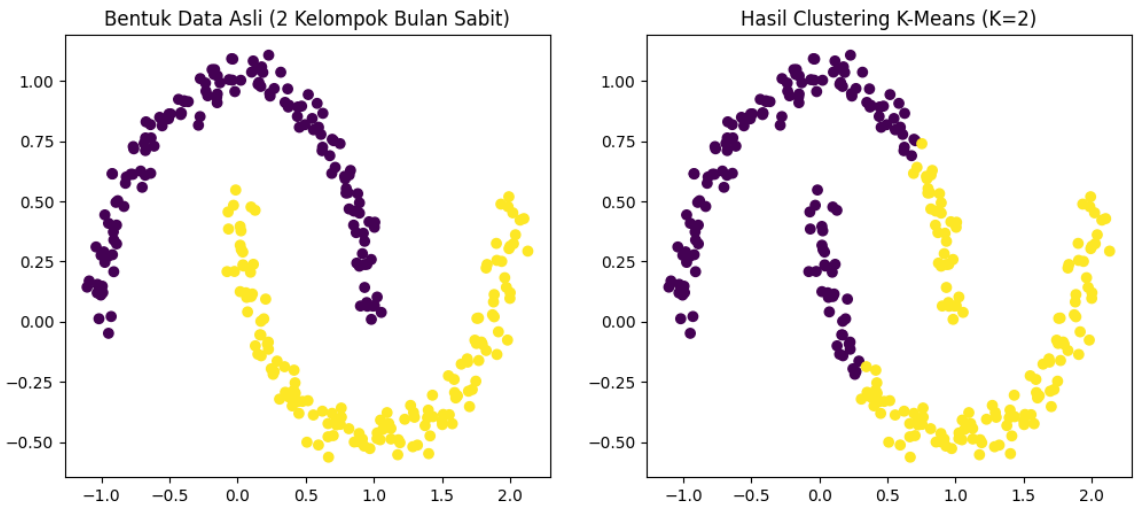
Langkah ini bersifat opsional, tergantung sisa waktu praktikum. Tujuannya untuk membuktikan secara visual keterbatasan K-Means yang sudah dibahas pada Dasar Teori bagian 2.2 dan 3 (Kekurangan): asumsi bentuk cluster spherical.

Dataset *Iris* memiliki *cluster* yang relatif bulat dan terpisah jelas, sehingga tidak menunjukkan kegagalan *K-Means* pada bentuk data yang lebih kompleks. Untuk itu, coba terapkan *K-Means* pada data sintetis berbentuk dua bulan sabit yang saling berkaitan:

```
# Membuat dataset sintetis berbentuk dua bulan sabit (non-
spherical)
X_moons, y_moons_true = make_moons(n_samples=300, noise=0.05,
random_state=42)

# Terapkan K-Means dengan K=2 (sesuai jumlah bentuk yang
terlihat)
kmeans_moons = KMeans(n_clusters=2, random_state=42, n_init=10)
labels_moons = kmeans_moons.fit_predict(X_moons)

# Bandingkan bentuk data asli vs hasil clustering K-Means
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].scatter(X_moons[:,0], X_moons[:,1], c=y_moons_true,
cmap='viridis')
axes[0].set_title('Bentuk Data Asli (2 Kelompok Bulan Sabit)')
axes[1].scatter(X_moons[:,0], X_moons[:,1], c=labels_moons,
cmap='viridis')
axes[1].set_title('Hasil Clustering K-Means (K=2)')
plt.show()
```



K-Means dijalankan dengan K=2 karena secara visual data memang membentuk dua kelompok bulan sabit, sehingga jumlah *cluster* bukan hal yang sedang diuji di sini. Fokus langkah ini adalah membandingkan bentuk data asli dengan cara *K-Means* membagi *cluster* berdasarkan jarak ke *centroid*, tanpa mempertimbangkan bentuk data yang sebenarnya.

K-Means membagi data menjadi dua kelompok yang dipisahkan oleh garis lurus, bukan mengikuti bentuk bulan sabit aslinya. Ini adalah bukti visual langsung bahwa *K-Means* kesulitan mengenali pola cluster yang bentuknya non-spherical, karena algoritma ini bekerja berdasarkan jarak Euclidean ke centroid (lihat Dasar Teori bagian 2.2).

Catatan Penting Praktikum

a. Sebelum menjalankan K-Means, pastikan data sudah distandarisasi.
b. Hasil clustering bisa berbeda jika centroid diinisialisasi secara berbeda; parameter n_init > 1 membantu mengurangi risiko ini dengan mencoba beberapa inisialisasi dan memilih hasil terbaik (lihat Langkah 6).
c. Parameter random_state digunakan agar proses acak (inisialisasi centroid) dapat direproduksi ulang secara konsisten setiap kali kode dijalankan.
d. Gunakan metode tambahan (misalnya Silhouette Score) untuk mengevaluasi kualitas cluster.
e. K-Means bekerja optimal pada cluster berbentuk bulat (spherical).

E. LATIHAN / STUDI KASUS PRAKTIKUM

Studi Kasus: Segmentasi Pelanggan Toko Retail

Link dataset : https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

Sebuah pusat perbelanjaan ingin membagi pelanggannya ke dalam beberapa segmen untuk meningkatkan efektivitas strategi pemasaran. Anda diberikan dataset Mall Customers yang berisi data pelanggan dengan atribut berikut:

a. CustomerID
b. Gender
c. Age
d. Annual Income (k$)
e. Spending Score (1–100)

Dengan menggunakan algoritma K-Means Clustering, lakukan analisis sebagai berikut:

1. Baca dataset *Mall Customers* ke dalam Python dan tampilkan 5 baris pertama.
2. Pilih fitur yang relevan untuk clustering, yaitu Annual Income (k$) dan Spending Score (1-100).
3. Lakukan standarisasi data sebelum diterapkan ke K-Means.
4. Tentukan jumlah cluster optimal dengan menggunakan Elbow Method.
5. Terapkan K-Means Clustering pada dataset dengan jumlah cluster hasil Elbow Method.
6. Visualisasikan hasil clustering dalam bentuk scatter plot dengan warna berbeda untuk tiap cluster.
7. Interpretasikan karakteristik tiap cluster yang terbentuk.
   - Misalnya: cluster dengan pendapatan tinggi dan spending score tinggi → segmen pelanggan premium.
   - Berikan nama segmen untuk tiap cluster.
8. Buat rekomendasi strategi pemasaran berdasarkan hasil clustering.